

面试手册

MySQL

把基础，读成底气。

1 个科目 / 20 道问题

回答要点 • 常见追问 • 易错点 • 参考来源

整理日期：2026-10-09

本手册总结经典面试知识点，不是全网公司面经的穷举，也不代表真实招聘频次统计。题解为独立中文整理，具体版本与实现请参考题目中的原始资料。

C++ 以 C++17/20 为主；MySQL 以 8.4 LTS InnoDB 为主；Redis 以开源版常用功能为主。目录、PDF 书签及参考链接均可点击。

阅读目录

阅读目录	2
MySQL	3
01. 为什么 InnoDB 使用 B+ 树索引？	3
02. 聚簇索引、二级索引与回表有什么关系？	3
03. 联合索引为什么有最左前缀原则？	4
04. 覆盖索引与索引条件下推有什么区别？	5
05. 哪些 SQL 写法可能降低索引利用率？	5
06. ACID 分别由什么机制保证？	6
07. MVCC 与 Read View 如何实现快照读？	7
08. RC 与 RR 的关键区别是什么？	8
09. RR 是否彻底解决幻读？	8
10. 记录锁、间隙锁与 next-key 锁怎么区分？	9
11. 死锁如何发生、排查与处理？	10
12. redo 与 undo 分别解决什么问题？	10
13. binlog 与 redo 如何通过两阶段提交协调？	11
14. 如何读懂 EXPLAIN 和 EXPLAIN ANALYZE？	12
15. JOIN 的语义与实现有哪些高频问题？	13
16. 慢 SQL 的排查步骤是什么？	13
17. 深分页为什么慢，游标分页怎么做？	14
18. 主从复制与读写分离有哪些一致性问题？	15
19. 范式、反范式与表设计如何取舍？	15
20. 如何防止 SQL 注入并管理数据库连接？	16

MySQL

索引、事务与查询优化

以 MySQL 8.4 LTS 与 InnoDB 为基准，覆盖从 B+ 树与 MVCC 到慢 SQL、复制一致性和表设计的高频追问。

01. 为什么 InnoDB 使用 B+ 树索引？

索引与存储 / 基础

InnoDB 普通索引采用面向页的 B 树结构，面试通常称为 B+ 树。高扇出降低树高，叶子保存索引记录并支持有序遍历，因此同时适合等值定位和范围查询。

回答要点

1. 页是磁盘与缓冲池管理的重要单位，一个内部节点可容纳大量键和子页指针。相同记录数下，多路树比二叉树层数更少，能减少随机访问页面的次数，而不是简单靠比较次数取胜。
2. 范围查询先定位起点，再按叶子记录的顺序向后扫描。哈希结构擅长等值查找，却无法天然支持有序输出、范围条件和前缀查找，因此不适合作为通用关系表的唯一索引结构。
3. 普通索引和空间索引不能混为一谈，空间索引使用 R 树。默认索引页大小为 16KB，但实例初始化可选择其他支持的页大小，树高还取决于键长度、记录大小以及页面填充情况。

常见追问

追问：索引越多越快吗？不是。每次插入、删除及索引列更新都要维护相关索引，并占用存储和缓冲池。应根据高频查询、选择性与写入比例保留有效索引，用实际计划和负载验证收益。

易错点

不要背出固定的“几层支持多少亿行”作为保证。树高由页大小与键宽度等共同决定，数据库命中缓冲池后，也不一定每层都产生磁盘 I/O。

延伸阅读 / 原始参考资料

MySQL 8.4 • 索引物理结构

02. 聚簇索引、二级索引与回表有什么关系？

索引与存储 / 基础

InnoDB 聚簇索引叶子保存行数据，二级索引记录包含索引列及主键。通过二级索引找到主键后再定位整行称为回表，覆盖索引可以在适用时减少这一步。

回答要点

1. 有显式主键时采用主键作为聚簇索引；没有主键时优先选所有列非空的第一个唯一索引；仍没有合适索引时，InnoDB 生成隐藏递增行标识。因此“没有主键就没有聚簇索引”错误。
2. 二级索引使用主键定位聚簇记录，而非保存一个永久的物理文件地址。主键越宽，各二级索引附带的主键空间通常越大，所以设计主键要兼顾稳定性、长度与插入访问模式。
3. 自增主键通常让插入更集中，减少随机插入造成的页面分裂，但也可能形成写入热点。随机 UUID 作为主键会增大键宽并分散写入，是否可接受应由真实规模与业务需求判断。

常见追问

追问：SELECT id WHERE email=? 会回表吗？若 email 二级索引已包含所需的 id，通常可以覆盖查询。覆盖描述的是所需列可由索引提供，但 MVCC 可见性判定等情况仍可能需要访问聚簇记录。

易错点

“聚簇”描述数据与索引组织方式，不代表表文件按主键连续无缝排列。也不能把任何数据库的二级索引都解释为携带主键，这是 InnoDB 的具体实现。

延伸阅读 / 原始参考资料

MySQL 8.4 • 聚簇与二级索引

MySQL 8.4 • 多版本机制

03. 联合索引为什么有最左前缀原则？

索引与存储 / 基础

联合索引按列顺序进行字典序排列。索引 (a, b, c) 可直接利用 a、(a, b)、(a, b, c) 的连续前缀定位；列顺序应围绕常用过滤、范围及排序需求设计。

回答要点

1. 先按 a 排序，a 相同时按 b，再按 c 排序。没有 a 条件时，b 的值分散在不同 a 区间内，不能把 b 条件一般化为一个连续索引范围，所以常规定位需要前导列约束。
2. 等值条件通常能继续利用后续列，范围条件会限制后续列进一步缩小扫描区间的能力。但后续列仍可能用于索引条件过滤、覆盖和排序判断，不能说遇到范围后就完全失效。
3. WHERE 条件书写顺序通常不决定索引使用顺序，优化器会分析谓词。MySQL 还有索引跳跃扫描等优化，因此缺少最左列并不等于绝对不能访问该索引，需要查看执行计划。

常见追问

追问：(status, created_at, id) 怎么选？若主要查询某状态按时间翻页，该顺序可将状态等值、时间范围与稳定排序结合。若主要按时间跨状态查询，前导 status 反而不适合，应基于查询占比调整或增设索引。

易错点

不要把“选择性最高的列永远放第一”当唯一规则。相等条件、范围条件、排序、覆盖、写入成本与查询组合都需要一起考虑。

延伸阅读 / 原始参考资料

MySQL 8.4 • 联合索引

MySQL 8.4 • 范围访问

04. 覆盖索引与索引条件下推有什么区别？

索引与存储 / 进阶

覆盖索引让查询所需列由索引提供，减少取整行；索引条件下推则让存储引擎先用索引中的列过滤，再决定是否读取整行。二者解决的阶段不同，可共同降低访问成本。

回答要点

1. 例如二级索引 (city, age) 可提供查询 city、age 和主键所需的值。如果还要读取 name 且它不在索引中，就仍需要取得整行，覆盖应结合 SELECT 投影与过滤条件具体判断。
2. ICP 把可由索引列检查的部分条件交给存储引擎执行。它不一定减少索引记录扫描数量，但能在获取完整行之前丢弃不满足条件的记录，从而降低随机读取聚簇页的成本。
3. 计划中的 Using index 通常提示覆盖访问，Using index condition 提示索引条件下推。它们不是同义词，也不等于查询已经最优，还应结合实际扫描行数、返回量与执行时间。

常见追问

追问：为什么 SELECT * 不利于优化？它增加传输和取行需求，让小而精确的覆盖索引难以成立；表结构变化还会改变查询输出。应列出业务真正需要的列，再判断是否值得加入索引，避免为了覆盖而堆宽索引。

易错点

不要认为 ICP 直接把二级索引变成覆盖索引。ICP 只是提前过滤，符合条件且缺少投影列的记录仍要取整行。

延伸阅读 / 原始参考资料

MySQL 8.4 • 索引条件下推

MySQL 8.4 • 执行计划字段

05. 哪些 SQL 写法可能降低索引利用率？

索引与存储 / 基础

索引能否有效使用取决于谓词可定位性、类型匹配、索引列序以及成本估计。对列做变换、前置通配符或返回大量行，常使普通索引无法高效定位，但不能一概称为索引失效。

回答要点

1. 对普通索引列套函数或表达式，例如 `DATE(created_at)=日期`，通常不如半开时间范围容易建立范围访问。若确实需要表达式查询，可评估生成列或函数索引，并检查版本限制。
2. 字符串列与数字比较等隐式类型转换，可能改变比较语义和访问路径；不同字符集或排序规则也会影响连接。应让参数类型与列定义一致，避免把类型问题误认为数据量太大。
3. `LIKE 'abc%'` 在合适条件下可构建索引范围，`LIKE '%abc'` 通常不能使用普通 B 树快速定位。即便可以访问索引，优化器也可能因返回比例高而认为表扫描成本更低。

```
-- 避免在普通索引列上套 DATE()
WHERE created_at >= '2026-10-09'
AND created_at < '2026-10-10'
```

常见追问

追问：OR 一定不用索引吗？不一定。优化器可能使用 Index Merge 或其他访问方式。应分别观察各分支的索引、选择性和合并成本，必要时比较等价的 UNION 写法，保证去重与 NULL 语义一致。

易错点

不要用“索引失效清单”代替验证。全索引扫描也属于使用索引，但未必高效；索引统计信息、数据分布和查询投影都可能影响最终计划。

延伸阅读 / 原始参考资料

MySQL 8.4 • 范围访问

MySQL 8.4 • EXPLAIN

06. ACID 分别由什么机制保证？

事务与并发 / 基础

ACID 是原子性、一致性、隔离性和持久性。InnoDB 通过撤销、日志、并发控制与恢复机制提供事务能力；业务一致性还依赖约束和正确的事务逻辑，不能只靠开启事务获得。

回答要点

1. 原子性要求事务修改整体提交或撤销，undo 支持回滚；持久性依赖 redo、提交刷盘策略及可靠存储。改变刷盘参数可能提高吞吐，但也改变进程故障、系统故障下的数据丢失窗口。
2. 隔离性由隔离级别、MVCC 与锁共同决定，不是所有事务都完全串行。数据库允许的并发异常随级别变化，应用需要根据业务约束选择锁定读、原子更新或更强隔离策略。
3. 一致性可以用“转账前后余额总和保持、库存不能为负”解释。数据库维护内部结构与声明的约束，应用则必须把扣款和入账纳入同一事务，并检查条件更新结果与失败处理。

常见追问

追问：提交成功就绝不丢数据吗？要结合日志刷盘参数、操作系统与存储设备保证讨论。严格持久性通常需提交时同步 redo 与 binlog，设备还必须真实兑现持久写入；跨机器故障切换另有复制一致性问题。

易错点

事务不会自动修复错误业务逻辑。例如先读取库存再无条件写入，仍可能丢失更新。先说明具体不变量，再选择原子条件更新或锁定策略。

延伸阅读 / 原始参考资料

MySQL 8.4 • ACID 模型

MySQL 8.4 • 撤销日志

07. MVCC 与 Read View 如何实现快照读？

事务与并发 / 进阶

MVCC 通过行版本与 undo 历史，在不阻塞写入的情况下提供一致性视图。Read View 根据事务可见性判断版本，当前版本不可见时沿历史版本查找，而不是复制整张数据库。

回答要点

1. 聚簇记录保存最近修改事务信息和回滚指针。更新产生可用于重建旧版本的 undo，读取时根据视图选择可见版本；自己的修改通常可见，所以快照并非对所有变化都一概隐藏。
2. RC 每次一致性读建立新的快照，RR 通常复用事务内第一次一致性读建立的快照。开始事务本身不等于普通情况下已经创建快照，WITH CONSISTENT SNAPSHOT 等语句要单独讨论。
3. 长时间运行的事务可能让旧版本持续被需要，阻碍 purge 清理并增大 undo 历史与空间压力。排查时要看活跃事务及历史积压，而不是只看当前 SQL 是否还在执行。

常见追问

追问：MVCC 能解决写写冲突吗？不能。快照读取减少读写互相阻塞，但多个事务修改同一行仍需锁协调。锁定读和更新使用与普通快照不同的读取语义，不能用某个旧快照直接推断更新会作用于哪些行。

易错点

不要说 Read View 是“当前最大事务号”一个数字。它包含活跃事务等可见性边界；也不要把 MVCC 等同于所有 SELECT 都不加锁。

延伸阅读 / 原始参考资料

MySQL 8.4 • 多版本机制

MySQL 8.4 • 一致性非锁定读

08. RC 与 RR 的关键区别是什么？

事务与并发 / 基础

InnoDB 默认 RR。RC 的一致性读逐语句刷新快照，RR 通常在同一事务复用首次快照；锁定范围和间隙锁行为也不同，因此选择隔离级别同时涉及可见性、并发与业务约束。

回答要点

1. RC 可以看到前一条语句之后其他事务已提交的修改，所以同一行重复读取可能变化。RR 普通一致性读稳定于事务快照，但自己的写入可见，混合快照读与当前读需要谨慎解释。
2. RR 对许多范围锁定操作使用 next-key 锁来限制区间插入。RC 对一般搜索和索引扫描关闭间隙锁，但外键约束检查和重复键检查仍可能使用，不能说 RC 完全没有间隙锁。
3. 更高隔离级别可能增加等待、降低并发，但也简化某些约束推理。选择 RC 不能免除事务、唯一约束或库存校验；选择 RR 也不能自动防止跨多行的所有业务并发异常。

常见追问

追问：降低到 RC 能解决死锁吗？它可能减少范围锁竞争，但记录锁、外键检查和不同更新顺序仍可构成死锁。应先分析锁依赖、缩短事务并统一访问顺序，再根据业务允许的异常决定是否调整隔离级别。

易错点

不要只背“四级隔离表”回答 InnoDB 的具体行为。SQL 标准描述允许的异常，存储引擎实现还涉及快照、锁定读和索引扫描范围。

延伸阅读 / 原始参考资料

MySQL 8.4 • 事务隔离级别

09. RR 是否彻底解决幻读？

事务与并发 / 进阶

要区分读取方式：RR 普通快照读用稳定视图避免看到其他事务后来插入的行；范围锁定读可用 next-key 阻止相关插入。混合快照读、当前读及自身更新，不能简单断言所有场景结果永远不变。

回答要点

1. 如果事务反复执行普通 SELECT，且未改变自身数据，第一次快照之后其他事务提交的新行通常不可见。这是可见性控制，并不是把查询条件对应的所有区间都加锁禁止插入。
2. SELECT ... FOR UPDATE 和 UPDATE 等需要读取并锁定当前状态。RR 范围搜索通过记录和间隙锁约束相关区间，能阻止其他事务插入满足锁定范围的新记录，锁范围由访问路径决定。

3. 先快照查询为空，随后当前读可能看到其他事务已提交的新记录；事务若更新这些新记录，自己的修改又会对后续快照查询可见。因此分析必须明确语句顺序、读类型和自身写入。

常见追问

追问：先查询不存在再插入如何防重复？优先建立正确的唯一约束，并处理重复键错误。若还需控制某范围的业务规则，可在合适索引上使用锁定读或设计约束表，不能依赖一次普通 SELECT 的空结果保证无人插入。

易错点

不要把“RR 的快照读没有新行”说成“RR 下其他事务不能插入”。快照控制能看见什么，范围锁控制能修改什么，是两种机制。

延伸阅读 / 原始参考资料

MySQL 8.4 • 幻影行与 next-key

MySQL 8.4 • 锁定读

10. 记录锁、间隙锁与 next-key 锁怎么区分？

事务与并发 / 进阶

记录锁锁索引记录，间隙锁限制区间插入，next-key 锁同时覆盖记录及其之前的间隙。InnoDB 行锁建立在索引上，实际范围取决于隔离级别、谓词、索引唯一性与执行路径。

回答要点

1. 记录锁针对索引中的某条记录；即使表没有显式索引，InnoDB 也会借助隐藏聚簇索引实现。共享锁与排他锁的兼容性不同，锁定读和普通一致性读也不能混为一种访问。
2. 间隙锁主要阻止往间隙插入，多个事务的间隙锁可以共存。next-key 把记录锁与前方间隙锁合在一起，范围搜索可能锁到边界附近，不能只按 WHERE 最终返回的行数估计锁数量。
3. 使用完整唯一键等值匹配到已存在的唯一记录时，通常只需记录锁；未命中、非唯一搜索或只使用复合唯一键部分列时不能套用该结论。缺少合适索引会扩大扫描和锁定影响范围。

常见追问

追问：意向锁有什么作用？事务在持有行共享或排他锁前，先声明对应的表级意向锁，帮助表锁与行锁兼容性检查。意向锁本身通常不是把整张表的每一行锁住，它与记录锁是不同层级。

易错点

不要说“没用索引就自动升级成表锁”。通常是扫描并锁定大量索引记录造成近似全表阻塞，而非一定发生数据库意义上的锁升级。

延伸阅读 / 原始参考资料

MySQL 8.4 • InnoDB 锁

11. 死锁如何发生、排查与处理？

事务与并发 / 进阶

死锁是多个事务形成循环等待。InnoDB 通常检测并回滚一个事务打破循环，应用必须能重试；预防重点是统一资源访问顺序、缩短事务并降低锁定范围，超时与死锁要分开分析。

回答要点

1. 例如事务甲先锁行 A 再等 B，事务乙先锁 B 再等 A，就构成循环。锁的数量少也会死锁，行锁、间隙、外键和不同索引访问组合都可能参与，不只有大批量更新才发生。
2. 通过 `SHOW ENGINE INNODB STATUS` 查看最近死锁，也可开启全部死锁日志并结合锁等待信息。把 SQL、索引、隔离级别、锁模式及事务内先前语句连起来，重建实际等待关系。
3. 让批量事务按稳定主键顺序更新，事务中避免远程调用和用户等待，并给条件加合适索引。重试整个事务应有次数上限和退避，外部副作用要幂等，不能只机械重发最后一条 SQL。

```
SHOW ENGINE INNODB STATUS;
```

常见追问

追问：锁等待超时会回滚整个事务吗？默认通常只回滚超时语句，死锁受害者则回滚整个事务；`innodb_rollback_on_timeout` 等配置会改变超时行为。应用应明确失败后的事务状态并执行回滚或整体重试。

易错点

死锁检测关闭后可能依赖等待超时解开阻塞，这不等于死锁消失。不要用无限重试掩盖持续的锁顺序问题，也不要把所有等待都判定为死锁。

延伸阅读 / 原始参考资料

MySQL 8.4 • 死锁

MySQL 8.4 • 错误处理

12. redo 与 undo 分别解决什么问题？

日志与恢复 / 基础

redo 记录支持崩溃恢复所需的修改信息，让已提交变化可重放；undo 保存撤销和旧版本信息，用于事务回滚与一致性读。二者职责不同，修改 undo 页本身也需要受到恢复保护。

回答要点

1. 数据库先把修改相关的 redo 按日志规则持久化，再逐步刷新脏数据页，通常称为先写日志。顺序日志写入比每次提交同步所有随机数据页更高效，重启可从检查点之后恢复必要变化。

2. undo 用于恢复修改前的逻辑状态，更新形成的历史还服务于 MVCC。事务提交后不代表所有 undo 立即删除，仍被旧快照需要的版本必须保留，最终由 purge 等后台机制清理。
3. redo 具有有限容量并不断推进检查点，日志空间压力可能迫使脏页刷新；undo 生命周期与事务和快照有关。解释两类日志时，应同时说明恢复、回滚和空间回收三个维度。

常见追问

追问：有 redo 还要 doublewrite 吗？redo 记录变化，但损坏或部分写入的数据页可能无法作为可靠恢复基础。doublewrite 保护页面写入完整性，帮助处理 torn page，与 redo 的逻辑职责不同；具体配置受设备原子写能力影响。

易错点

不要把 redo 称为“SQL 原文日志”，也不要说 undo 只在失败时有用。提交成功后，其他事务的快照读仍可能依赖历史版本。

延伸阅读 / 原始参考资料

MySQL 8.4 • 重做日志

MySQL 8.4 • 撤销日志

MySQL 8.4 • 双写缓冲

13. binlog 与 redo 如何通过两阶段提交协调？

日志与恢复 / 进阶

binlog 属于 MySQL 服务层，支持复制与时间点恢复；redo 属于 InnoDB，支持引擎崩溃恢复。提交协调让引擎状态与 binlog 事务记录一致，通常用 prepare、写 binlog、commit 来概括。

回答要点

1. row 格式记录行变化，statement 格式记录语句，mixed 按规则选择。复制和恢复依赖 binlog，但它不能替代 InnoDB redo 对本地数据页的恢复职责，日志格式也会影响体积与可重现性。
2. 引擎先进入准备状态，随后 binlog 提交记录持久化，再完成引擎提交。恢复时检查准备事务与有效 binlog 的事务标识，决定完成提交还是回滚，避免两份日志对同一事务意见相反。
3. 概念上的三个步骤并不意味着每笔事务都独自刷盘三次，组提交可合并同步。严格讨论持久性时，需要结合 sync_binlog 与 innodb_flush_log_at_trx_commit，并考虑操作系统和硬件故障。

常见追问

追问：只开启 binlog 就能恢复到任意时间吗？需要一份可靠的一致性全量备份，以及从该备份起连续可用的 binlog。恢复先装载备份，再重放到目标事务边界；日志被清理或时间范围有缺口就无法任意回放。

易错点

不要把内部提交协调等同于应用显式发出的分布式 XA 全流程。面试可以用两阶段图解释核心，但要承认组提交与恢复规则使实现比背诵的三行顺序更复杂。

延伸阅读 / 原始参考资料

MySQL 8.4 • 二进制日志

MySQL 8.4 • 内部与外部 XA

14. 如何读懂 EXPLAIN 和 EXPLAIN ANALYZE?

查询优化 / 基础

EXPLAIN 展示优化器选择的访问方式和估计；EXPLAIN ANALYZE 实际执行可分析语句并提供耗时、行数和循环。应比较估计与实际，而不是只看是否出现某个索引或某个 Extra 标签。

回答要点

1. 观察 join 顺序、访问 type、possible_keys 与实际 key、估计 rows 和 filtered。rows 是估计扫描量，并非结果条数；多层循环需要结合各层循环次数判断总工作量，不能孤立比较单个数字。
2. Using filesort 表示未直接用索引顺序完成排序，不等于必然写磁盘；Using temporary 也需要看数据量和实现。小表扫描可能合理，索引随机回表太多时甚至比顺序扫描更贵。
3. EXPLAIN ANALYZE 对照实际耗时、返回行和 loops，有助发现统计偏差与重复访问。因为会实际执行并消耗资源，应在可控环境或合适负载时分析，不能当成零成本的静态展示。

```
EXPLAIN FORMAT=TREE
SELECT id FROM orders WHERE user_id=42;

EXPLAIN ANALYZE
SELECT id FROM orders WHERE user_id=42;
```

常见追问

追问：计划选错索引怎么办？先核对参数类型、数据分布、统计信息和索引设计，可比较更新统计后的计划与真实耗时。强制索引可以做诊断对比，但把它长期写死可能在数据分布变化后阻止优化器选择更好路径。

易错点

type 的传统顺序是提示而非性能总排名。一次 const 查询也可能被上层重复百万次；一次 ALL 扫描小表则可能非常快，必须看整体工作量。

延伸阅读 / 原始参考资料

MySQL 8.4 • EXPLAIN

MySQL 8.4 • 计划字段

15. JOIN 的语义与实现有哪些高频问题？

查询优化 / 基础

INNER JOIN 保留匹配组合，LEFT JOIN 还保留左侧未匹配行并补 NULL。物理执行可使用嵌套循环、索引查找或哈希连接，MySQL 8.4 的实现不能照搬旧版本只有嵌套循环的结论。

回答要点

1. 对 LEFT JOIN 的右表列在 WHERE 中施加排除 NULL 的条件，常使未匹配行被过滤，结果等价于某些内连接。若想保留全部左行，应明确条件应放 ON 还是 WHERE，并验证业务语义。
2. 索引嵌套循环利用外侧结果逐次查找内侧索引，适合外侧筛选后较少的情况。哈希连接可先构建哈希表再匹配另一侧，内存不足可能使用临时文件，具体计划取决于代价估计。
3. 连接键数据类型和排序规则应一致，先减少可安全下推的输入数据，并避免无意的一对多结果膨胀。常说小表驱动大表，更准确应指过滤后的有效输入及访问成本，而非原始总行数。

常见追问

追问：IN、EXISTS、JOIN 哪个更快？没有固定赢家。优化器可能把子查询转换为半连接等同类执行路径。先保证重复行和 NULL 语义等价，再对真实参数与数据查看计划及实际耗时，不要只按语法名称决定性能。

易错点

不要认为 LEFT JOIN 一定比 INNER JOIN 慢，也不要为了性能随意把外连接改为内连接。两者结果语义不同，优化首先必须保持正确结果。

延伸阅读 / 原始参考资料

MySQL 8.4 • 哈希连接

MySQL 8.4 • 嵌套循环连接

MySQL 8.4 • 外连接简化

16. 慢 SQL 的排查步骤是什么？

查询优化 / 进阶

先确认慢发生在执行、锁等待、网络还是连接队列，再定位高耗时或高频 SQL，检查计划与实际扫描量。优化应围绕最大成本，并在同样负载与参数下比较，不能只加索引或盲目加机器。

回答要点

1. 结合慢查询日志、Performance Schema 和应用链路，检查总耗时、锁等待、扫描与返回行数。单次快但频率极高的查询也可能消耗主要资源，只筛最长的一条会遗漏整体吞吐瓶颈。
2. 核对是否扫描过多、排序聚合过大、重复回表、连接结果膨胀或存在热点锁。优先缩小查询范围、只取必要列和调整索引，再考虑缓冲池、缓存或架构变化，避免优化不在瓶颈上的环节。

3. 评估 P95/P99 延迟、吞吐及写入代价，使用有代表性的参数和冷热缓存。线上加索引还要评估磁盘、DDL 并发和部署窗口，改写 SQL 必须检查重复、NULL、排序与事务语义。

常见追问

追问：为何缓存命中后仍慢？可能在锁等待、CPU 排序、扫描大量缓存页或应用传输解析上耗时。内存命中只减少磁盘读取，不能消除查询计算与竞争，应先用执行统计确认具体耗时位置。

易错点

慢日志的 Query_time 与端到端响应时间并不相同，也不是所有日志字段都包含完整等待。不要用一次测试的最快耗时证明优化有效。

延伸阅读 / 原始参考资料

MySQL 8.4 • 慢查询日志

MySQL 8.4 • 缓冲池

17. 深分页为什么慢，游标分页怎么做？

查询优化 / 进阶

LIMIT offset,size 往往要访问并丢弃前 offset 条，页数越深成本越高。游标分页以最后一条排序键作为下一页边界，用索引直接继续扫描，适合顺序浏览，但不提供任意页码跳转。

回答要点

1. 即使排序用到了索引，较大的 OFFSET 仍可能需要扫描大量记录，若伴随回表成本更高。可以先用覆盖索引拿目标主键再取行，但它降低取行成本，不会消除遍历深层偏移的本质。
2. 按 created_at,id 的稳定唯一顺序分页时，游标记录上一页末尾的两个值，下一页使用字典序边界并同样排序。id 用于解决时间相同的并列，否则会漏读或重复，方向也必须一致。
3. 并发插入或更新排序键可能影响跨页结果。若需要导出一致的完整集合，应考虑事务快照、固定上界或版本标识；普通无限滚动与严格审计导出不能使用同一种一致性假设。

```
SELECT id, created_at
FROM orders
WHERE user_id = 42
  AND (created_at > ? OR (created_at = ? AND id > ?))
ORDER BY created_at, id
LIMIT 20;
-- 对应索引: (user_id, created_at, id)
```

常见追问

追问：游标分页能直接跳第 1000 页吗？通常不能，需要已有边界或预先维护定位信息。它把按页码随机访问换成高效顺序推进；产品若必须跳页，可限制最大页数、另建检索接口或接受偏移查询成本。

易错点

不要只写 `WHERE id > last_id` 却继续按另一列排序。游标条件、索引顺序和 `ORDER BY` 必须协调，同时把唯一的并列键纳入边界。

延伸阅读 / 原始参考资料

MySQL 8.4 • LIMIT 优化

MySQL 8.4 • 排序优化

18. 主从复制与读写分离有哪些一致性问题？

复制与建模 / 进阶

常规 MySQL 复制由主库 binlog 向副本传送并应用，默认异步，因此副本可能落后。读写分离增加读吞吐，却可能让用户写后立即读到旧值；半同步确认也不等于副本已应用完成。

回答要点

1. 副本接收主库日志形成 relay log，再由应用线程执行，网络接收与 SQL 应用是不同阶段。大事务、慢查询争用、磁盘或网络压力都会增加延迟，判断一致性需要知道事务是否已经应用。
2. 需要写后读一致时，可短时间或按会话路由到主库，也可依据 GTID 等事务位置等待副本追上并设置超时回退。固定睡眠只能降低概率，无法在负载和故障变化下给出可靠证明。
3. 半同步让主库等待副本接收并持久记录所需事件的确认，降低特定故障下的丢失风险，但超时可回退异步且确认不等于副本完成应用。故障切换仍需选主、日志完整性与旧主隔离策略。

常见追问

追问：副本越多写入越快吗？不会。写入通常仍由主库承担，更多副本还带来日志传输和管理成本。副本适合扩展容忍一定延迟的读；写热点和容量瓶颈可能需要优化事务、分区或更复杂的数据分片方案。

易错点

不要把“复制线程在线”当成“数据实时一致”。Seconds_Behind_Source 等延迟指标也不是所有故障下的完美证明，应结合日志位置、应用状态与业务一致性要求。

延伸阅读 / 原始参考资料

MySQL 8.4 • 半同步复制

MySQL 8.4 • GTID 复制

19. 范式、反范式与表设计如何取舍？

复制与建模 / 基础

范式通过约束函数依赖减少冗余和更新异常；反范式为读性能或历史语义有意保存重复数据。设计应先表达实体、键与约束，再根据查询成本决定冗余，并明确唯一事实来源和更新责任。

回答要点

1. 面试可先解释一范式避免重复组，二范式去除非主属性对候选键的部分依赖，三范式关注非主属性的传递依赖。分析依据是候选键与函数依赖，而非简单按列数或是否只有单列主键判断。
2. 订单项保存成交时价格和商品名称快照，通常属于历史事实，即使商品表以后变化也不应自动更新。把它误判为应消除的重复字段，会破坏历史语义，冗余是否错误必须结合业务含义。
3. 读多写少的聚合可维护汇总表以降低实时连接和统计成本，但需定义更新事务、幂等重放、补偿与校验。没有同步责任的重复字段会产生不一致，不能只为了少 JOIN 就复制所有数据。

常见追问

追问：为何不全部用 JSON 一列存储？JSON 适合结构灵活的局部属性，但核心筛选、连接、唯一约束与关系仍需要清晰建模。大量字段隐藏在文档中会增加索引、类型校验和数据治理成本，应按访问与约束需求选型。

易错点

不要认为三范式永远比反范式快，也不要把加一个代理 id 视为自动消除函数依赖问题。候选键及业务依赖仍然存在，模型应能够解释更新异常。

延伸阅读 / 原始参考资料

MySQL 8.4 • 数据大小与规范化

RPI CSCI 4380 • 数据库规范化课程

20. 如何防止 SQL 注入并管理数据库连接？

复制与建模 / 基础

SQL 注入的根源是把不可信输入拼入 SQL 语法。使用参数化查询绑定值，动态表名或排序列采用白名单；再配合最小权限、正确连接池与超时控制，既保护数据也避免资源耗尽。

回答要点

1. 参数绑定让用户输入作为数据而非语法解释，适用于值参数。表名、列名和排序方向通常不能用值占位符代替，应从预设选项映射为可信标识符，不能把它们直接拼接进查询。
2. 参数化之外，还需限制数据库账号权限、避免应用使用管理员账号，并检查日志不要输出密码或敏感数据。转义和输入格式校验可作为辅助，但不能替代驱动支持的参数绑定。
3. 连接池复用会话以减少建立连接成本，但必须归还连接、回滚未结束事务并重置会话状态。连接数过多会增加服务器压力，应同时设置连接等待、执行与事务期限并观测排队情况。

-- 值由驱动绑定，禁止字符串拼接

```
SELECT id, name FROM users WHERE email = ?;
```

常见追问

追问：ORM 就一定安全吗？普通参数化接口通常安全，但原生 SQL、字符串拼接及动态排序仍可能绕过保护。应检查最终调用是否真正绑定值，动态标识符是否使用白名单，事务异常后连接是否带着未提交状态返回池中。

易错点

不要把正则过滤引号当完整防护。输入有多种编码和语法边界，正确的查询构造、参数绑定及权限隔离比黑名单更可靠。

延伸阅读 / 原始参考资料

MySQL 8.4 • 预处理语句

MySQL 8.4 • PREPARE